Javascript : BOM et DOM

JavaScript API, Browser Object Model et interface Window

Dans cette nouvelle partie, nous allons définir ce qu'est une API et expliquer en détail comment fonctionnent certaines API et notamment celles intégrées dans les navigateurs qui vont nous permettre, entre autres, de manipuler le code HTML ou les styles d'une page.

Définition et présentation des API JavaScript ¶

Une API (Application Programming Interface ou Interface de Programmation Applicative en français) est une interface, c'est-à-dire un ensemble de codes grâce à laquelle un logiciel fournit des services à des clients.

Le principe et l'intérêt principal d'une API est de permettre à des personnes externes de pouvoir réaliser des opérations complexes et cachant justement cette complexité.

En effet, le développeur n'aura pas besoin de connaître les détails de la logique interne du logiciel tiers et n'y aura d'ailleurs pas accès directement puisqu'il devra justement passer par l'API qui va nous fournir en JavaScript un ensemble d'objets et donc de propriétés et de méthodes prêtes à l'emploi et nous permettant de réaliser des opérations complexes.

Une API peut être comparée à une commande de voiture (pédale d'accélération, essuie-glace, etc.) : lorsqu'on accélère ou qu'on utilise nos essuies glace, on ne va pas se préoccuper de comment la voiture fait pour effectivement avancer ou comment les essuies glace fonctionnent. On va simplement se contenter d'utiliser les commandes (l'API) qui vont cacher la complexité des opérations derrière et nous permettre de faire fonctionner la voiture (le logiciel tiers).

Les API JavaScript vont pouvoir être classées dans deux grandes catégories :

- Les API intégrées aux navigateurs web et qu'on va donc pouvoir utiliser immédiatement pour du développement web comme l'API DOM (Document Object Model) qui va nous permettre de manipuler le HTML et le CSS d'une page, l'API Geolocation qui va nous permettre de définir des données de géolocalisation ou encore l'API Canvas qui permet de dessiner et de manipuler des graphiques dans une page ;
- Les API externes, proposées par certains logiciels ou sites comme la suite d'API Google Map qui permettent d'intégrer et de manipuler des cartes dans nos pages web ou encore l'API Twitter qui permet d'afficher une liste de tweets sur un site par exemple ou bien l'API YouTube qui permet d'intégrer des vidéos sur un site.
- Dans ce cours, nous allons nous concentrer sur les API générales et qui n'ont pas besoin de passer par des services tiers, c'est-à-dire sur les API intégrées aux navigateurs web qui sont rassemblées dans ce qu'on appelle le BOM (Browser Object Model).

> *Les API vont fonctionner en définissant un ou plusieurs objets qui vont nous fournir des propriétés et des méthodes nous permettant de réaliser des opérations complexes.*

Browser Object Model (BOM)

Le BOM est une sorte de « super API » elle-même composée de plusieurs API dont certaines sont elles mêmes composées de plusieurs API et etc.

A la base du BOM, nous avons l'interface Window qui représente une fenêtre de navigateur contenant une page ou un document. L'objet Window implémente l'interface Window. Cet objet est supporté par tous les navigateurs et tous les objets globaux, variables globales et fonctions globales appartiennent automatiquement à cet objet (c'est-à-dire sont des enfants de cet objet).

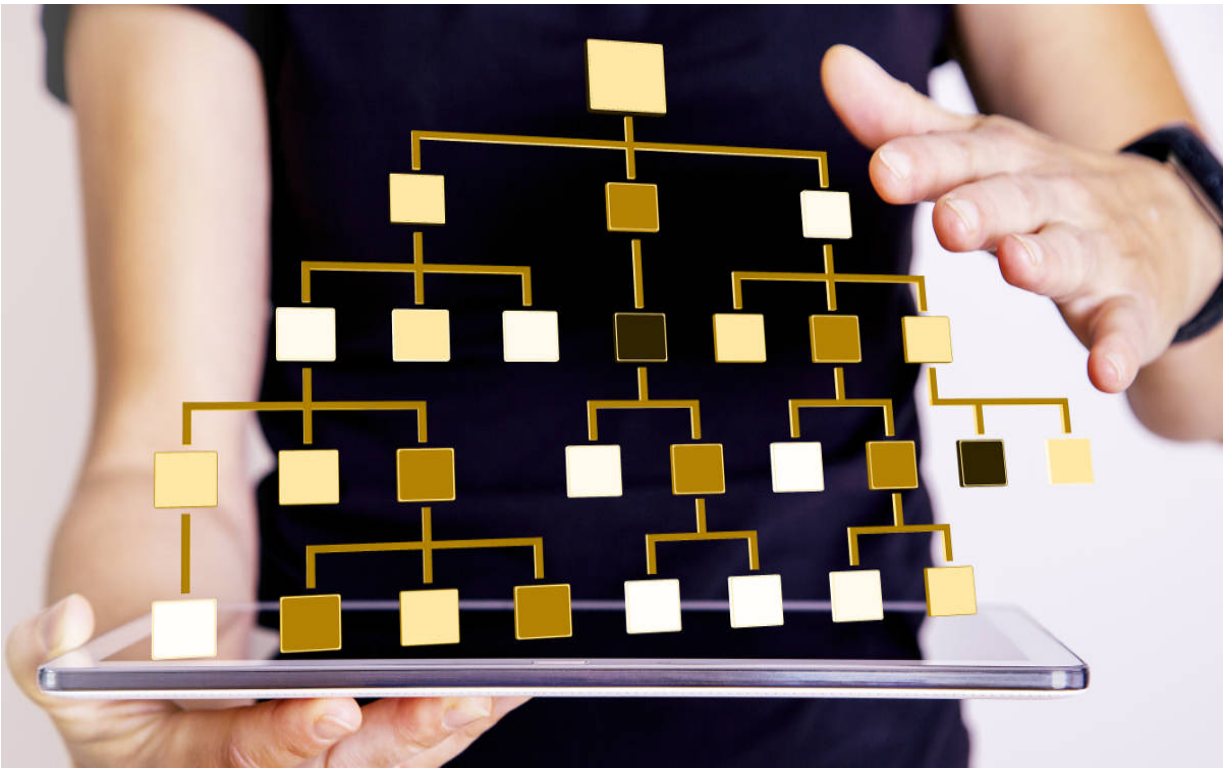
Dans un navigateur utilisant des onglets, comme Firefox, chaque onglet contient son propre objet Window. Cet objet Window est un objet dit « implicite » : nous n'aurons généralement pas besoin de le mentionner de manière explicite pour utiliser les méthodes (ou fonctions globales) et propriétés (ou variables globales) lui appartenant.

Le BOM est composé de différentes interfaces qu'on va pouvoir utiliser via des objets. Dans la suite de cette partie, nous ne parlerons plus que d' « objets » par simplicité même si ce terme ne sera pas toujours strictement exact.

Les objets suivants appartiennent au BOM et sont tous des enfants de Window :

- L'objet **Navigator** qui représente l'état et l'identité du navigateur et qu'on va utiliser avec l'API Geolocation ;
- L'objet **History** qui permet de manipuler l'historique de navigation du navigateur
- L'objet **Location** qui fournit des informations relatives à l'URL de la page courante ;
- L'objet **Screen** qui nous permet d'examiner les propriétés de l'écran qui affiche la fenêtre courante ;
- L'objet **Document et le DOM** dans son ensemble que nous étudierons en détail dans la suite.

Document Object Model (DOM)



Qu'est-ce que le Document Object Model (DOM) ?

Le Document Object Model (en français modèle d'objets de document), abrégé en DOM, est une interface de programmation normalisée permettant de structurer des documents HTML et XML. Cette interface a été développé et publiée par le World Wide Web Consortium (W3C), un organisme fondé en 1994 par l'inventeur du Web, Tim Berners-Lee, dans le but de développer des normes destinées au World Wide Web.

L'objectif du Document Object Model est d'offrir aux programmeurs un accès aux différents composants d'un projet Web, pour ainsi faciliter au mieux l'insertion, la suppression ou l'édition des contenus, d'attributs et de styles. DOM fait office de plateforme indépendante et de liaison (neutre quant au langage de programmation) entre les scripts de programmation comme JavaScript et le document Web de base, en affichant la structure du document sous forme d'une arborescence, dans laquelle chaque nœud représente un objet indépendant et configurable. En raison de la forme de cette structure, cette manière de représenter un projet Web est appelé un DOM-tree (arbre DOM en français).

Note :

Contrairement à ce que pourrait laisser entendre le nom, le DOM n'est pas un modèle, mais comme nous l'avons déjà dit, il s'agit plutôt d'une interface de programmation. Par extension, un Document Object Model peut cependant être considéré comme un modèle qui définit la manière dont on peut accéder aux données du Web, représentées comme un objet.

La plupart des clients Web utilise le DOM ou des interfaces basées sur le DOM, pour générer des pages HTML ou XML. Dans ce processus, les différents composants sont réunis en tant que nœuds, et organisés en arbre DOM. Parallèlement, le navigateur en question charge le rendu du document Web dans la mémoire locale, afin d'en assurer l'analyse, un éventuel traitement, pour pouvoir afficher ensuite la page sous la forme qui a été définie par le concepteur du site. Pour le rendu, les navigateurs utilisent différents moteurs (Rendering-Software), comme Gecko (Firefox), Webkit (Safari) ou Blink (Chrome, Edge, Opera) qui tous, s'appuient sur le standard DOM.

Le Document Object Model reste pertinent comme présentation orientée objet d'un document Web. Il a aussi son intérêt comme interface pour l'ensemble des contenus dynamiques programmés, et par conséquent pour toutes les interactions servant à modifier l'aspect d'une page pendant sa création.

Comment est structuré le Document Object Model ?

Des balises HTML définissent les relations entre les différents tags. Les éléments d'un document Web, identifiés par des tags, ont un niveau inférieur ou supérieur, en fonction de leur rôle dans le projet Web. Certains tags peuvent d'ailleurs être renfermés dans d'autres tags. Pour restituer correctement ces hiérarchies dans le Document Object Model, l'interface a recours à la structure en arbre que nous avons déjà évoquée, et qui permet de préciser le rendu de l'objet.

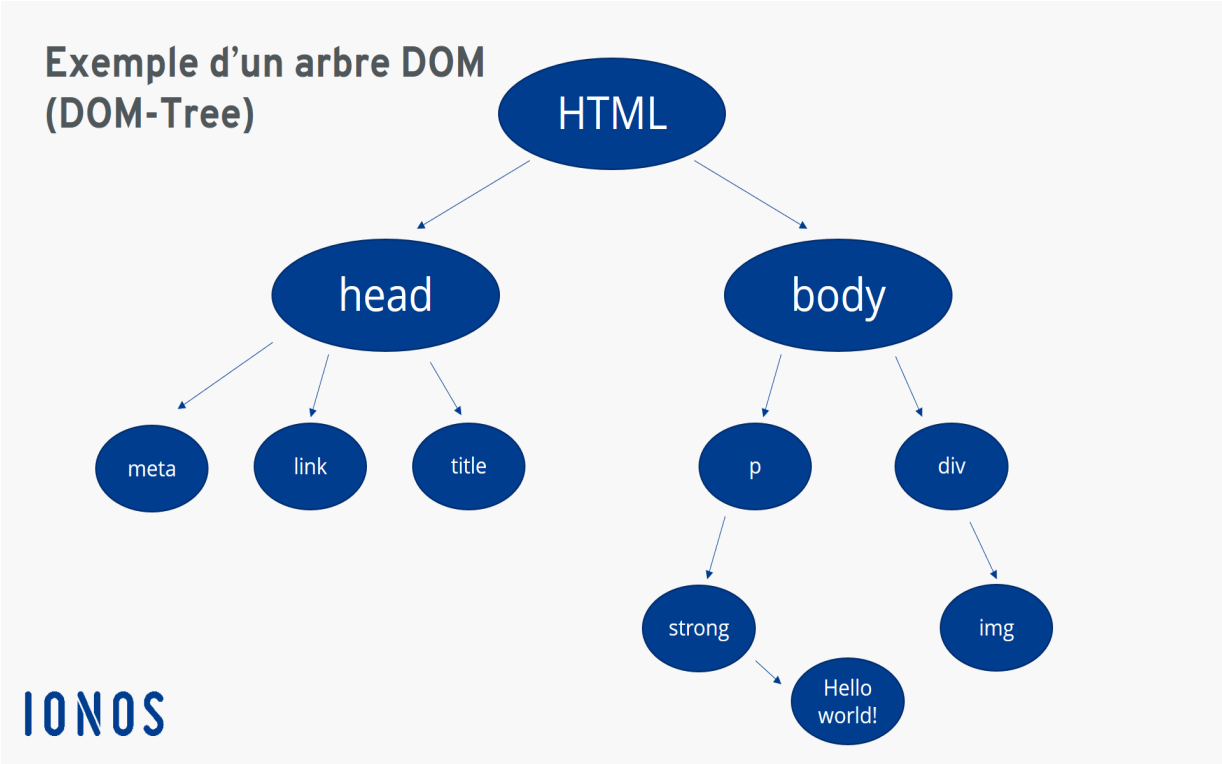
Note: Les différents éléments d'un DOM-Tree sont appelés des Nodes ou des Nœuds. Entre les différents nœuds, comme les objets Body, Header et Paragraph ci-dessus, on distingue des nœuds d'attribut, tels que « align » ou « size » et des nœuds de texte.

La manière dont un DOM-Tree est construit, dépend toujours du document HTML ou XML qui est à la base. Pour les documents HTML, nous retiendrons la hiérarchie de base suivante, et qui couvre les projets.

Le Document Object Model : un exemple pratique (incluant le code HTML)

Maintenant que nous avons examiné dans les rubriques précédentes la structure générale du Document Object Model, nous allons, au moyen d'un exemple concret, présenter quelques techniques de structuration élémentaires applicables aux scripts de programmation. La base est le document HTML suivant, avec Header - incluant des liens vers les feuilles de style CSS - et le message principal, ainsi que deux éléments de contenu (image et texte) :

```
<!DOCTYPE html>
<html>
<head>
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <link href="style.css" rel="stylesheet">
    <title>Title</title>
</head>
<body>
    <p><strong>Hello World!</strong></p>
    <div><img src="bild.jpg"></div>
</body>
</html>
```



Les objets Node ou nœuds du DOM

Le terme « nœud » est un terme générique qui sert à désigner tous les objets contenus dans le DOM. A l'extrémité de chaque branche du DOM se trouve un nœud.

A partir du nœud racine qui est le nœud HTML on voit que 2 branches se forment : une première qui va aboutir au nœud HEAD, une deuxième qui aboutit à un nœud un nœud BODY.

De nouvelles branches se créent ensuite à partir des nœuds HEAD et BODY et etc.

Les types de nœuds du DOM

On va pouvoir distinguer les nœuds les uns des autres en fonction de s'il s'agit d'un nœud constitué par un texte, par un élément, par un commentaire, etc. On va pouvoir utiliser des propriétés et méthodes différentes avec chaque type de nœud puisqu'ils vont dépendre d'interfaces différentes.

Pour être tout à fait précis, voici les différents types de nœuds que vous pourrez rencontrer et qui sont représentés par des constantes auxquelles une valeur est liée :

Constante	**Valeur**	**Description**
ELEMENT_NODE	1	Représente un nœud élément (comme p ou div par exemple)
TEXT_NODE	3	Représente un nœud de type texte
PROCESSING_INSTRUCTION_NODE	7	Nœud valable dans le cas d'un document XML. Nous ne nous en préoccuperons pas ici.
COMMENT_NODE	8	Représente un nœud commentaire
DOCUMENT_NODE	9	Représente le nœud formé par le document en soi
DOCUMENT_TYPE_NODE	10	Représente le nœud doctype
DOCUMENT_FRAGMENT_NODE	11	Représente un objet document minimal qui n'a pas de parent (ce type de nœud ne nous intéressera pas ici

Par ailleurs, vous pouvez noter qu'il existait auparavant d'autres constantes de type nœuds qui sont aujourd'hui dépréciées dont notamment ATTRIBUTE_NODE (représentant l'attribut d'un élément) et d'autres constantes liées au XML qui ne nous intéressent pas ici. Ces constantes ne doivent plus être utilisées mais il est possible que vous les rencontriez toujours sur certains sites.

L'un des intérêts majeurs du DOM et des nœuds va être qu'on va pouvoir se déplacer de nœuds en nœuds pour manipuler des éléments en utilisant le JavaScript.

Ce qui suit est une brève liste des API communes au script de page dans le Web et XML utilisant le DOM.

- document.getElementById(id)
- document.getElementsByTagName(name)
- document.createElement(name)
- parentNode.appendChild(node)
- element.innerHTML
- element.style.left
- element.setAttribute()
- element.getAttribute()
- element.addEventListener()
- window.content
- window.onload (en-US)
- console.log()
- window.scrollTo()

https://developer.mozilla.org/fr/docs/Web/API/Document_Object_Model

soit :

```
<!DOCTYPE html>
<html>
  <head>
    <title>JavaScript DOM Methods</title>
  </head>
  <body>
    <div id="container" class="container">
      <nav id="menu" class="navbar">
            <ul class="nave-items">
               <li class="nave-item">
                  <a class="btn link"> Home </a>
               </li>
               <li class="nave-item">
                  <a class="btn link"> Our Services </a>
                <li class="nave-item">
                  <a class="btn link"> Contact </a>
                </li>
                <li class="nave-item">
                  <a class="btn link"> About Us </a>
                </li>
            </ul>
        </nav>
      <section class="left">
         <article class="wrapper">
            <h3> Our Services </h3>
            <p id="message">A paragraph</p>
            <form id="contactForm" class="form">
                <input type="radio" name="language" value="JavaScript">
                <input type="radio" name="language" value="TypeScript">
                <input type="submit name="submit" value="Submit">
            </form>
        </article>
      </section>
    </div>
  </body>
</html>
```

```
// document.getElementById
const p =document.getElementById('message');
console.log(p);


// document.getElementsByClassName
/*
  let elements = document.getElementsByClassName(classNames);
  let elements = parentElement.getElementsByClassName(classNames);
*/
let menuItem =document.getElementsByClassName('btn link');


// document.getElementsByName(name)
let choice = document.getElementsByName('language');

// element.className
let menu =document.querySelector('#menu');
console.log(menu.className);
menu.className += newClassName;
```